

EE 705: VLSI Design Lab

Course Project Proposal

February 28, 2026

Team Members

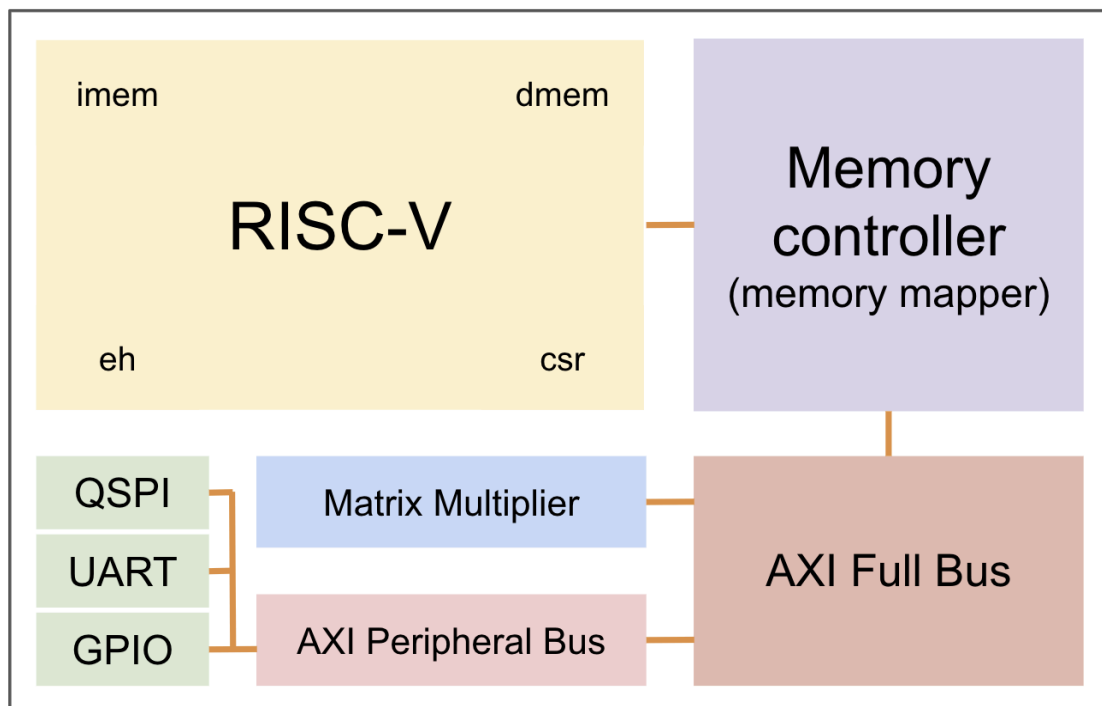
Kaarmukilan B	23b1215
Santhosh Kumar B	23b1221
Tamilaruvi S	23b1286
Athav Vantan B	23b1306
Dikshant Mendhe	25m1223
Hari Vikinesh	25m1229
Krishna N Atreya	25m1231

Objective

Design and Implementation of **pipelined** RV32IM and 8x8 systolic array matrix multiplier accelerator. Additionally, UART and QSPI will be implemented for the serial interface. Along with that, a BOOT ROM is added for reprogrammability.

Architecture

The idea is to build the project as close to a small microcontroller. The top-level view of the design is shown in the following figure.



RISC-V CPU

The CPU design will be a 5-stage **pipeline** design along with dedicated instruction memory and data memory. The CPU will also have an exception handler and CSR updater. Along with that, the CPU Design will have a two-bit branch predictor to improve IPC by reducing the structural hazards.

Extension Supported: fullset of i, m, zicntr extensions, subset of zicsr

Memory Mapping Table (mapping may vary over time)

Address Range	Memory Segments
0x0002_0000 - 0x0005_FFFF	Dedicated h/w accelerator space
0x0001_1000 - 0x0001_FFFF	Peripheral Memory
0x0001_0000 - 0x0001_0FFF	Data Memory
0x0000_2000 - 0x0000_FFFF	Program Memory
0x0000_0000 - 0x0000_1FFF	BOOT ROM

Below are the details and usage of the different memory regions.

- **BOOT ROM** - It contains the reset_vector, UART programmer initializer, and TRAP Handlers. This will be written by us in C, compiled, and loaded into the ROM.
- **Program Memory** - It will contain the firmware from the programmer.
- **Data Memory** - This is the space for the runtime code segments.
- **Peripheral Memory** - Each peripheral will have its own configuration, status, and data locations.
- **Dedicated H/W Accelerator** - This region is a dedicated space for the Matrix Multiplier accelerator.

Memory Controller

The memory controller will be the AXI Full Master, fully aware of the memory mapping, and will handle all communication between the CPU and the Slave devices. The memory controller will also have a linear/circular buffer of the required size to sequentialize the traffic.

AXI Buses

AXI Buses, like AXI4 Full and AXI Peripheral Bus, will be the primary way of communicating with the slave devices (UART, QSPI, GPIO).

AXI Peripheral Bus will have flip-flop synchronizers, handshakers with gray coders as described in the standards.

Matrix Multiplier

The systolic array matrix multiplier will take reference from the following paper and parallelize as much as possible for the remaining resources. <https://link.springer.com/article/10.1007/s11265-020-01582-7>

The Overall flow of the design is as follows.

1. The Bootloader runs from the BOOT ROM at the start of the program.
 - 1.1. It will put the CPU to sleep until the program is loaded into memory
 - 1.2. The program loads into the device in the following way.
UART -> APB -> AXI4 -> Memory Controller -> Memory
2. The CPU executes the code from the memory.
 - 2.1. If there are exceptions, the CPU traps them and resolves them from the Interrupt Handler Vector in the Boot ROM.
3. Any peripheral can be accessed using the memory addresses. The dedicated registers are used to configure them.
4. Use the Hardware Accelerator,
 - 4.1. The CPU loads the data into the dedicated memory section
 - 4.2. Updates the Matrix Multiplier configuration register
 - 4.3. Check the status register for completion.
 - 4.4. Displays the result through UART.

Timeline

Here is the tentative plan shown in the simplified Gantt Chart, subject to change.

Design subsets	Week1 Mar 3- 6	Week2 Mar 10-13	Week3 Mar 16-20	Week4 Mar 24-27	Week5 Mar31-Apr3	Week6 Apr7-Apr10
RV32 I						
RV32 M						
Matrix Mul						
zicsr						
Bootloader						
Peripherals						
AXI Buses						
Interfacing						

Demonstration Plan

The demonstration plan to verify the design's operation is as follows.

1. The Bootloader loads the C Program into the memory through the PC UART.
2. The C Program is written to test the following.
 - a. Make an LED Blink
 - b. Print texts from the UART to the PC
 - c. Working of the Matrix Multiplier

Week-wise Plan

Here's a week-by-week plan to complete the entire project:

Week 1: Showcase the implementation of RV32I in simulation.

Week 2: Demonstrate the working of the multiplier, including the divisor and reminder operations. (RV32M)

Week 3: Showcase the working of the matrix multiplier as an individual module.

Week 3: Discuss the idea for the zicsr and bootloader implementation. Also, demonstrate the peripheral implementations.

Week 4: Demonstrate the implementations of zicsr and AXI Buses.

Week 5: Demonstrate the working of the bootloader and assemble the project.

Week 6: Complete the demonstration as discussed in the previous sections.